Абстракция

Абстракция – парадигма, позволяющая на этапе проектирования ответить на вопрос: “что должен делать объект”.[1]

Абстракция — это выделение главных, существенных характеристик объекта и его внешнего интерфейса с сокрытием внутренних деталей реализации. Она позволяет сформулировать единый “контракт” (набор методов), которому обязаны следовать все его конкретные подклассы.[2]

В Python абстракция реализуется с помощью встроенного модуля `ABC` (Abstract Base Classes):

Особенности:

1. **Создание абстрактного класса:** Класс наследуется от `abc.ABC`.
2. **Абстрактные методы:** Методы интерфейса, у которых нет готовой реализации, помечаются декоратором `@abc.abstractmethod`.
3. **Защита от инстанцирования:** Создать экземпляр абстрактного класса напрямую нельзя. Если в подклассе не реализован хотя бы один абстрактный метод, при попытке создать его экземпляр Python выбросит исключение `TypeError`.

```python
from abc import ABC, abstractmethod
import math

class Shape(ABC):

    @abstractmethod
    def area(self):
        """Метод для вычисления площади"""
        pass

    @abstractmethod
    def perimeter(self):
        """Метод для вычисления периметра"""
        pass


class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

[1] определение из лекции по ООП от 10.10.26

[2] Объектно-ориентированное мышление – Мэтт Вайсфельд

```
    def perimeter(self):
        return 2 * (self.width + self.height)


class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius
```

```
rect = Rectangle(5, 10)
circle = Circle(7)
```

Площадь прямоугольника:

```
rect.area()
```

```
50
```

Периметр прямоугольника:

```
rect.perimeter()
```

```
30
```

Площадь круга:

```
circle.area()
```

```
153.93804002589985
```

Вот что будет, если попробовать создать экземпляр абстрактного класса:

```
shape = Shape()
```

```
Traceback (most recent call last):
  File "<chunk>", line 1, in <module>
TypeError: Can't instantiate abstract class Shape without an implementation for abstract methods 'area
```